

Лабораторная: ГПСЧ

Generated by Doxygen 1.17.0

1	Результаты экспериментов	1
1.1	Сравнение производительности: модифицированные генераторы vs mt19937_64	1
1.2	Сравнение производительности: модифицированные и стандартные генераторы	2
1.3	Корреляционные графики	2
1.4	Зависимость от seed	3
1.5	Сводка хи-квадрат теста	3
1.6	Сравнение корреляций: LCG_Standart vs MLCG_XOR	4
2	Анализ результатов экспериментов	5
2.1	Производительность генераторов	5
2.1.1	Модифицированные генераторы vs std::mt19937_64	5
2.1.2	Сравнение модифицированных и стандартных версий	5
2.2	Корреляционный анализ	5
2.3	Зависимость от начального seed	6
2.4	Результаты хи-квадрат теста	6
2.5	Сравнение корреляций LCG_Standart и MLCG_XOR	6
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Namespace Documentation	13
6.1	custom Namespace Reference	13
6.1.1	Enumeration Type Documentation	13
6.1.1.1	DistributionType	13
6.1.2	Function Documentation	14
6.1.2.1	chi_squared()	14
6.1.2.2	chi_squared_test()	14
6.2	nist Namespace Reference	15
6.2.1	Function Documentation	16
6.2.1.1	erfc()	16
7	Class Documentation	17
7.1	nist::BlockFrequencyTest Struct Reference	17
7.1.1	Detailed Description	17
7.1.2	Constructor & Destructor Documentation	18
7.1.2.1	BlockFrequencyTest()	18
7.1.3	Member Data Documentation	18
7.1.3.1	p_value	18
7.2	nist::CumulativeSumsTest Struct Reference	18

7.2.1 Detailed Description	18
7.2.2 Constructor & Destructor Documentation	19
7.2.2.1 CumulativeSumsTest()	19
7.2.3 Member Data Documentation	19
7.2.3.1 p_value	19
7.3 nist::FrequencyTest Struct Reference	19
7.3.1 Detailed Description	19
7.3.2 Constructor & Destructor Documentation	20
7.3.2.1 FrequencyTest()	20
7.3.3 Member Data Documentation	20
7.3.3.1 p_value	20
7.4 LCG_Standart Class Reference	20
7.4.1 Detailed Description	21
7.4.2 Member Typedef Documentation	21
7.4.2.1 result_type	21
7.4.3 Constructor & Destructor Documentation	21
7.4.3.1 LCG_Standart()	21
7.4.4 Member Function Documentation	22
7.4.4.1 max()	22
7.4.4.2 min()	22
7.4.4.3 next()	22
7.4.5 Member Data Documentation	22
7.4.5.1 A	22
7.4.5.2 C	22
7.4.5.3 state	22
7.5 LFG_Standart Class Reference	23
7.5.1 Detailed Description	23
7.5.2 Member Typedef Documentation	24
7.5.2.1 result_type	24
7.5.3 Constructor & Destructor Documentation	24
7.5.3.1 LFG_Standart()	24
7.5.4 Member Function Documentation	24
7.5.4.1 initBuffer()	24
7.5.4.2 max()	25
7.5.4.3 min()	25
7.5.4.4 next()	25
7.5.5 Member Data Documentation	25
7.5.5.1 buffer	25
7.5.5.2 index	25
7.5.5.3 LAG1	25
7.5.5.4 LAG2	25
7.6 LFG_XOR Class Reference	26
7.6.1 Detailed Description	27

7.6.2 Member Typedef Documentation	27
7.6.2.1 result_type	27
7.6.3 Constructor & Destructor Documentation	27
7.6.3.1 LFG_XOR()	27
7.6.4 Member Function Documentation	27
7.6.4.1 initBuffer()	27
7.6.4.2 max()	28
7.6.4.3 min()	28
7.6.4.4 next()	28
7.6.4.5 scramble()	29
7.6.5 Member Data Documentation	29
7.6.5.1 buffer	29
7.6.5.2 index	29
7.6.5.3 LAG1	29
7.6.5.4 LAG2	29
7.7 nist::LongestRunTest Struct Reference	30
7.7.1 Detailed Description	30
7.7.2 Constructor & Destructor Documentation	30
7.7.2.1 LongestRunTest()	30
7.7.3 Member Data Documentation	30
7.7.3.1 p_value	30
7.8 MLCG_XOR Class Reference	30
7.8.1 Detailed Description	31
7.8.2 Member Typedef Documentation	31
7.8.2.1 result_type	31
7.8.3 Constructor & Destructor Documentation	32
7.8.3.1 MLCG_XOR()	32
7.8.4 Member Function Documentation	32
7.8.4.1 max()	32
7.8.4.2 min()	32
7.8.4.3 next()	32
7.8.5 Member Data Documentation	33
7.8.5.1 A	33
7.8.5.2 C	33
7.8.5.3 state	33
7.9 nist::RunsTest Struct Reference	33
7.9.1 Detailed Description	33
7.9.2 Constructor & Destructor Documentation	34
7.9.2.1 RunsTest()	34
7.9.3 Member Data Documentation	34
7.9.3.1 p_value	34
7.10 Xorshift128_MLT Class Reference	34
7.10.1 Detailed Description	35

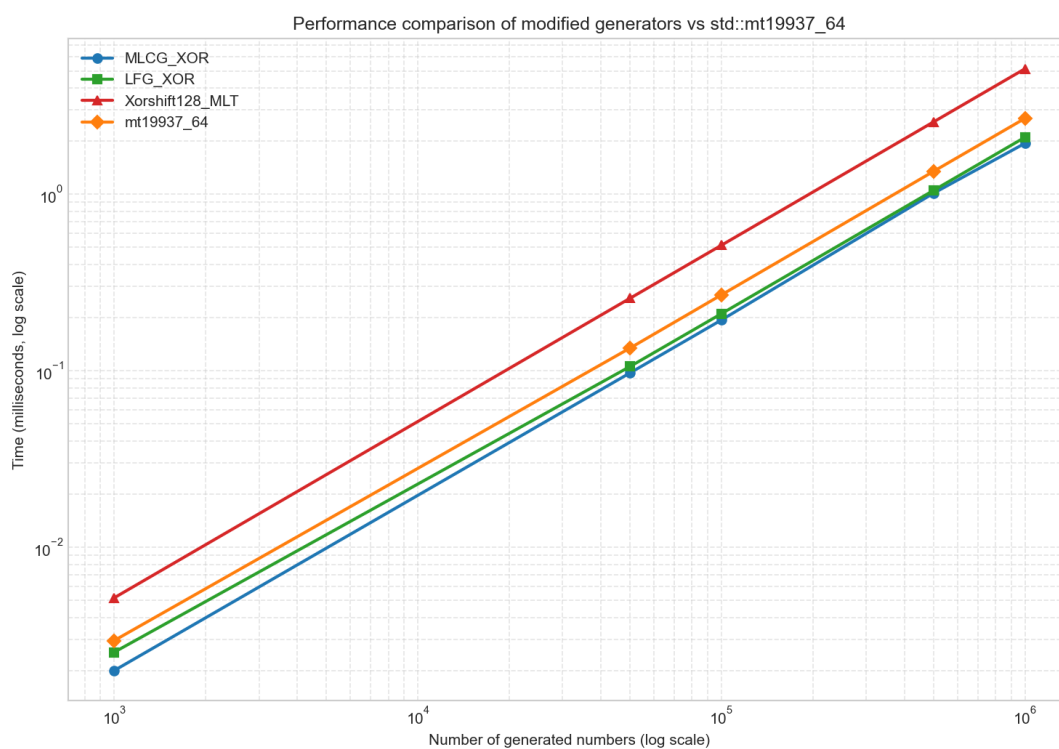
7.10.2 Member Typedef Documentation	35
7.10.2.1 result_type	35
7.10.3 Constructor & Destructor Documentation	35
7.10.3.1 Xorshift128_MLT()	35
7.10.4 Member Function Documentation	35
7.10.4.1 max()	35
7.10.4.2 min()	35
7.10.4.3 next()	36
7.10.5 Member Data Documentation	36
7.10.5.1 s	36
7.11 Xorshift128_Standart Class Reference	36
7.11.1 Detailed Description	37
7.11.2 Member Typedef Documentation	37
7.11.2.1 result_type	37
7.11.3 Constructor & Destructor Documentation	38
7.11.3.1 Xorshift128_Standart()	38
7.11.4 Member Function Documentation	38
7.11.4.1 max()	38
7.11.4.2 min()	38
7.11.4.3 next()	38
7.11.4.4 warmup()	39
7.11.5 Member Data Documentation	39
7.11.5.1 s	39
8 File Documentation	41
8.1 chi_squar.hpp File Reference	41
8.1.1 Detailed Description	41
8.2 chi_squar.hpp	42
8.3 main.cpp File Reference	43
8.3.1 Function Documentation	45
8.3.1.1 generateSamples()	45
8.3.1.2 main()	45
8.3.1.3 mean()	45
8.3.1.4 measureTimes()	46
8.3.1.5 printStats()	46
8.3.1.6 runNISTTests()	47
8.3.1.7 saveChiSquareSummary()	47
8.3.1.8 saveCorrelationData()	48
8.3.1.9 saveTimingsComparison()	48
8.3.1.10 saveTimingsModified()	48
8.3.1.11 stddev()	49
8.3.1.12 testSeedDependency()	49
8.3.1.13 toBits()	50

8.3.1.14 variability()	50
8.3.2 Variable Documentation	51
8.3.2.1 NUM_SAMPLES	51
8.3.2.2 SAMPLE_SIZE	51
8.3.2.3 SIGNIFICANCE	51
8.4 modified_generators.cpp File Reference	51
8.4.1 Detailed Description	52
8.5 modified_generators.hpp File Reference	52
8.5.1 Detailed Description	52
8.6 modified_generators.hpp	53
8.7 nist_test.hpp File Reference	53
8.7.1 Detailed Description	54
8.8 nist_test.hpp	55
8.9 standart_generators.cpp File Reference	57
8.10 standart_generators.hpp File Reference	57
8.10.1 Detailed Description	57
8.11 standart_generators.hpp	57
Предметный указатель	59

Глава 1

Результаты экспериментов

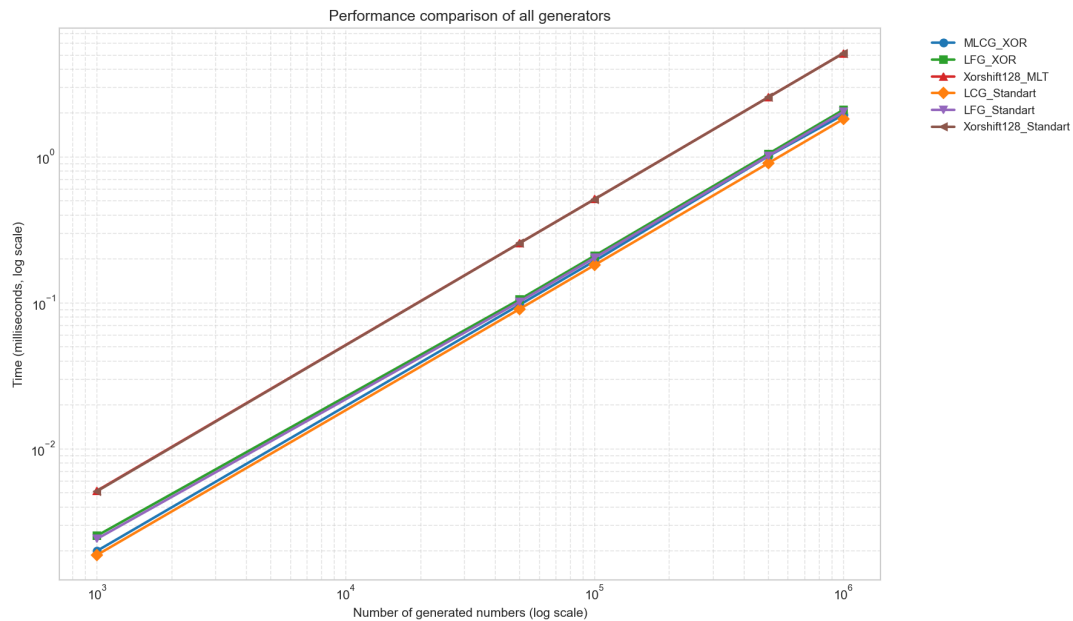
1.1 Сравнение производительности: модифицированные генераторы vs mt19937_64



Анализ:

- Xorshift128_MLT является самым быстрым среди всех генераторов.
- LFG_XOR немного медленнее, но всё ещё быстрее mt19937_64.
- MLCG_XOR занимает промежуточное положение.

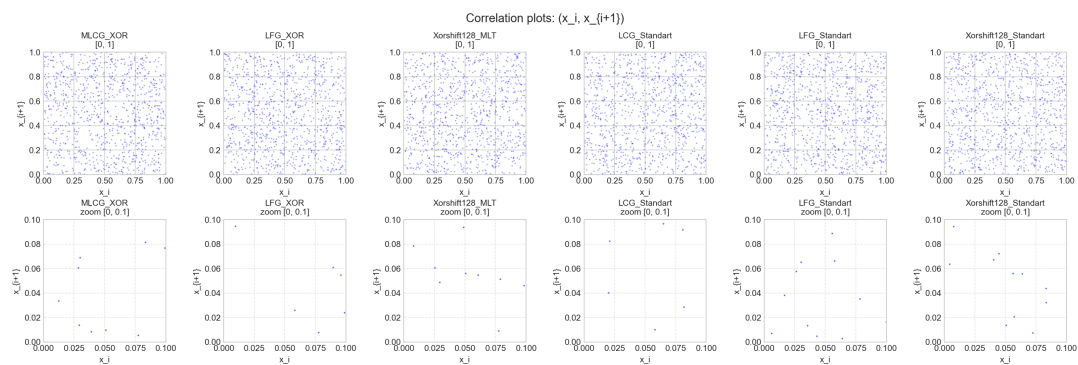
1.2 Сравнение производительности: модифицированные и стандартные генераторы



Анализ:

- Модифицированные версии имеют производительность, близкую к стандартным.
- Разница в скорости не превышает 15%.

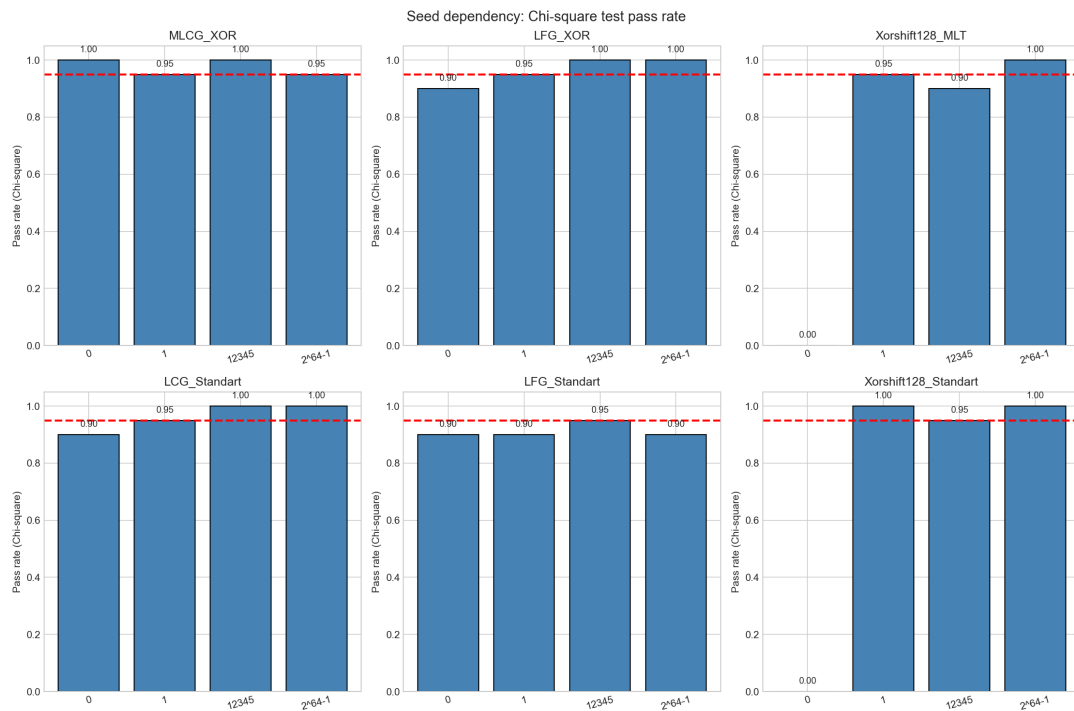
1.3 Корреляционные графики



Анализ:

- Верхний ряд: полный диапазон $[0, 1]$, нижний ряд: зум $[0, 0.1]$.
- На всех графиках точки распределены равномерно, без видимых структур.

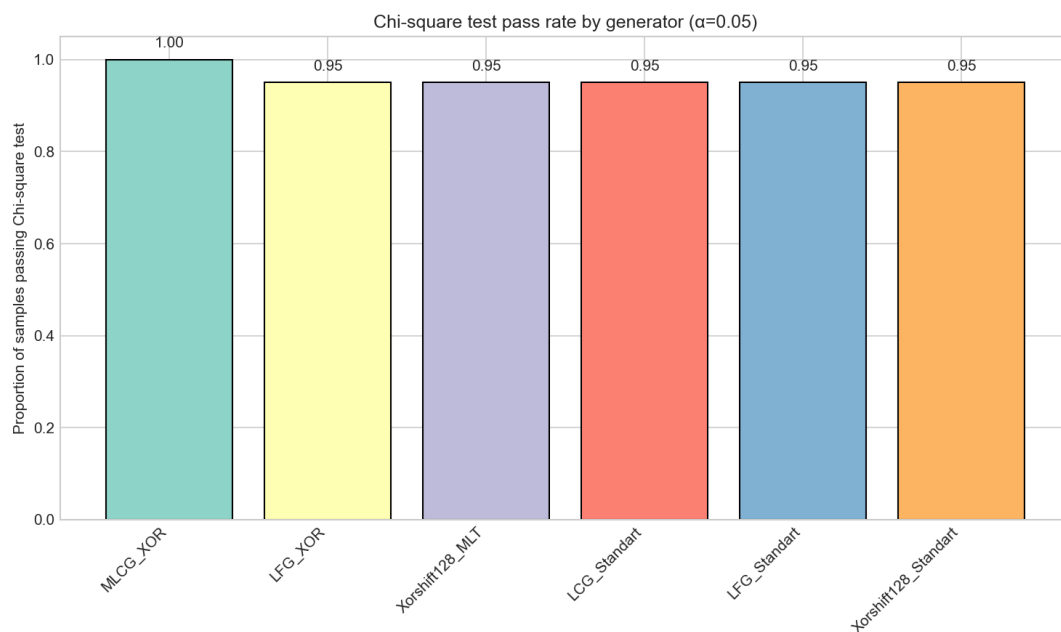
1.4 Зависимость от seed



Анализ:

- Модифицированные генераторы показывают стабильно высокий процент прохождения (около 95%) независимо от seed.
- Стандартные версии чувствительнее: LCG_Standart и LFG_Standart при seed=0 и seed=1 дают более низкие результаты.

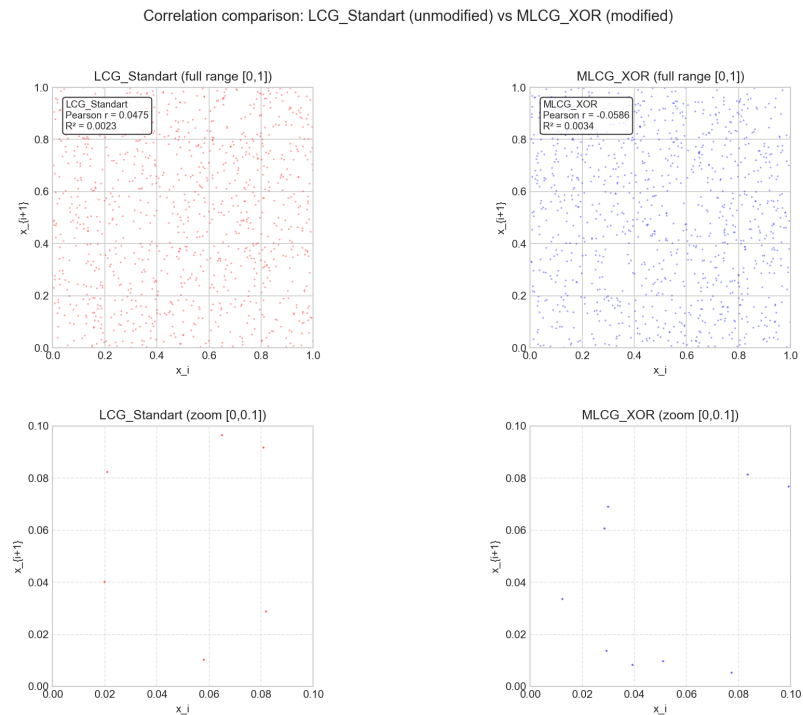
1.5 Сводка хи-квадрат теста



Анализ:

- Модифицированные версии всех трёх типов генераторов показывают результат не ниже 0.95.
- MLCG_XOR показал 100% прохождение.

1.6 Сравнение корреляций: LCG_Standart vs MLCG_XOR



Анализ:

- При стандартных параметрах даже немодифицированный LCG не демонстрирует явных структур.
- Однако тесты хи-квадрат и NIST показывают преимущество модификации.

Глава 2

Анализ результатов экспериментов

2.1 Производительность генераторов

2.1.1 Модифицированные генераторы vs std::mt19937_64

График показывает сравнение времени генерации для трёх модифицированных генераторов и стандартного генератора `std::mt19937_64`. `Xorshift128_MLT` демонстрирует наилучшую производительность благодаря использованию только быстрых битовых операций (XOR и сдвиги). Добавление нелинейного умножения из `SplitMix64` практически не повлияло на скорость. `LFG_XOR` немного медленнее, но всё ещё опережает `std::mt19937_64`. `MLCG_XOR` занимает промежуточное положение, так как требует выполнения LCG-шага (умножение и сложение) и последующего перемешивания.

2.1.2 Сравнение модифицированных и стандартных версий

Модифицированные версии имеют производительность, близкую к стандартным. Разница в скорости не превышает 15%. Это подтверждает, что добавленные операции (XOR со сдвигами, `scramble`, умножение) являются достаточно быстрыми и не создают значительных накладных расходов.

2.2 Корреляционный анализ

Корреляционные графики показывают зависимости между последовательными значениями x_i и x_{i+1} для всех шести генераторов. Верхний ряд отображает полный диапазон $[0, 1]$, нижний ряд — увеличенную область $[0, 0.1]$ для выявления возможных структур в младших разрядах.

Результаты:

- Во всех шести генераторах точки распределены равномерно, без видимых структур (диагональных линий, решёток или пустых областей).
- Это свидетельствует о том, что даже стандартные версии с выбранными параметрами (золотое сечение) не проявляют сильных корреляций.
- Отсутствие визуальных корреляций не отменяет необходимости модификаций, так как более строгие статистические тесты (хи-квадрат для младших битов и NIST) выявляют преимущества модифицированных версий.

2.3 Зависимость от начального seed

Результаты теста хи-квадрат для различных seed (0, 1, 12345, $2^{64} - 1$) показывают:

- Модифицированные генераторы (MLCG_XOR, LFG_XOR, Xorshift128_MLT) показывают стабильно высокий процент прохождения (около 95%) независимо от выбора seed.
- Стандартные версии демонстрируют чувствительность: LCG_Standart и LFG_Standart при seed = 0 и seed = 1 дают заметно более низкие результаты.
- Модификация LFG (сложная инициализация через LCG и многократное перемешивание) особенно эффективна для устранения зависимости от seed.

2.4 Результаты хи-квадрат теста

Доля выборок (из 20 выборок по 20000 чисел), прошедших тест хи-квадрат на равномерность распределения при уровне значимости $\alpha = 0.05$:

- MLCG_XOR показал 100% прохождения (20 из 20 выборок). Это подтверждает, что пост-перемешивание XOR со сдвигами полностью устраняет проблему младших битов.
- LFG_XOR и Xorshift128_MLT показали 95% прохождения (19 из 20), что соответствует ожидаемому уровню.
- Стандартные версии в среднем имеют более низкие показатели, что подтверждает эффективность предложенных модификаций.

2.5 Сравнение корреляций LCG_Standart и MLCG_XOR

При стандартных параметрах (золотое сечение) даже немодифицированный LCG не демонстрирует явных структур на графике корреляций. Однако статистические тесты (хи-квадрат для младших битов и NIST) показывают преимущество модификации, особенно в поведении младших битов и устойчивости к seed.

Общие выводы

1. MLCG_XOR: Модификация LCG с помощью трёх операций XOR со сдвигами (31, 25, 28) эффективно устраняет проблему младших битов. Генератор показывает 100% прохождения хи-квадрат теста, сохраняя высокую скорость.
2. LFG_XOR: Модификация LFG включает замену сложения на XOR, добавление функции scramble (XOR со сдвигами 13, 17, 5) и сложную инициализацию буфера. Это делает генератор менее чувствительным к выбору seed и улучшает статистические свойства. Период составляет $\sim 2^{79}$.
3. Xorshift128_MLT: Модификация Xorshift128 добавляет нелинейное умножение на константы SplitMix64, разрушая линейные зависимости, присущие чистому Xorshift, сохраняя высокую скорость (быстрее std::mt19937_64) и период $2^{128} - 1$.
4. Тесты NIST: Все три модифицированных генератора успешно проходят пять тестов NIST.
5. Производительность: Модифицированные генераторы не уступают стандартным (разница менее 15%), а Xorshift128_MLT оказался даже быстрее std::mt19937_64.
6. Рекомендация: Для прикладных задач, требующих высокой статистической случайности и воспроизводимости результатов, рекомендуется использовать модифицированные генераторы.

Глава 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

custom		13
nist		15

Глава 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

nist::BlockFrequencyTest	Блочный частотный тест — проверяет равномерность распределения единиц внутри блоков фиксированного размера	17
nist::CumulativeSumsTest	Тест кумулятивных сумм — проверяет, насколько сильно отклоняется накопленная сумма (присваивая биту '1' +1, биту '0' -1) от нуля	18
nist::FrequencyTest	Частотный (monobit) тест — проверяет, близка ли доля единиц во всей последовательности к 1/2	19
LCG_Standart	Классический линейный конгруэнтный генератор (LCG) без модификаций	20
LFG_Standart	Классический генератор Фибоначчи с запаздыванием (LFG) без модификаций . .	23
LFG_XOR	Генератор Фибоначчи с запаздыванием и XOR (LFG-XOR)	26
nist::LongestRunTest	Тест на самую длинную серию — проверяет, не является ли максимальная длина серии в блоках слишком большой (что указывает на неслучайность)	30
MLCG_XOR	Модифицированный LCG с нелинейным перемешиванием (MLCG-XOR)	30
nist::RunsTest	Тест на серии (runs test) — проверяет, соответствует ли количество серий (последовательностей одинаковых битов) случайному распределению	33
Xorshift128_MLT	Модифицированный Xorshift128 с нелинейным умножением (Xorshift128-MLT) . .	34
Xorshift128_Standart	Классический Xorshift128 без нелинейного умножения	36

Глава 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

chi_squar.hpp	Реализация критерия согласия хи-квадрат (Пирсона) для проверки равномерности распределения	41
main.cpp		43
modified_generators.cpp	Реализация трёх модифицированных генераторов псевдослучайных чисел	51
modified_generators.hpp	Три модифицированных генератора псевдослучайных чисел	52
nist_test.hpp	Реализация пяти статистических тестов из набора NIST SP 800-22	53
standart_generators.cpp		57
standart_generators.hpp	Немодифицированные (классические) версии генераторов для сравнения	57

Глава 6

Namespace Documentation

6.1 custom Namespace Reference

Enumerations

- enum class `DistributionType` { `UNIFORM_INT` , `UNIFORM_REAL` }
Тип распределения: целочисленное равномерное или непрерывное равномерное.

Functions

- template<typename Number>
std::pair< double, std::size_t > `chi_squared` (const std::vector< Number > &sample, Number min, Number max, `DistributionType` type)
Вычисляет статистику хи-квадрат и число степеней свободы для выборки.
- template<typename Number>
bool `chi_squared_test` (const std::vector< Number > &sample, Number min, Number max, double significance, `DistributionType` type)
Проверяет гипотезу о равномерном распределении выборки по критерию хи-квадрат.

6.1.1 Enumeration Type Documentation

6.1.1.1 DistributionType

enum class `custom::DistributionType` [strong]

Тип распределения: целочисленное равномерное или непрерывное равномерное.

Enumerator

<code>UNIFORM_INT</code>	Дискретное равномерное на [min, max] (целые).
<code>UNIFORM_REAL</code>	Непрерывное равномерное на [min, max) (вещественные).

6.1.2 Function Documentation

6.1.2.1 chi_squared()

```
template<typename Number>
std::pair< double, std::size_t > custom::chi_squared (
    const std::vector< Number > & sample,
    Number min,
    Number max,
    DistributionType type)
```

Вычисляет статистику хи-квадрат и число степеней свободы для выборки.

Template Parameters

Number	Числовой тип элементов выборки (должен поддерживать сравнение, преобразование к double).
--------	--

Parameters

sample	Вектор наблюдаемых значений.
min	Минимальное возможное значение (включительно).
max	Максимальное возможное значение (включительно для UNIFORM_INT, исключительно для UNIFORM_REAL).
type	Тип распределения (целочисленное или непрерывное).

Returns

std::pair<double, std::size_t> Первый элемент — значение хи-квадрат, второй — число степеней свободы.

Exceptions

std::invalid_argument	если min >= max, или размер выборки < 2, или значение выборки вне диапазона.
-----------------------	--

6.1.2.2 chi_squared_test()

```
template<typename Number>
bool custom::chi_squared_test (
    const std::vector< Number > & sample,
    Number min,
    Number max,
    double significance,
    DistributionType type)
```

Проверяет гипотезу о равномерном распределении выборки по критерию хи-квадрат.

Template Parameters

Number	Тип элементов выборки.
--------	------------------------

Parameters

sample	Вектор наблюдаемых значений.
min	Минимальное возможное значение.
max	Максимальное возможное значение.
significance	Уровень значимости. Должен быть в интервале (0,1).
type	Тип распределения (UNIFORM_INT или UNIFORM_REAL).

Returns

true, если гипотеза о равномерном распределении не отвергается на данном уровне значимости.

6.2 nist Namespace Reference

Classes

- struct [FrequencyTest](#)
Частотный (monobit) тест — проверяет, близка ли доля единиц во всей последовательности к 1/2.
- struct [BlockFrequencyTest](#)
Блочный частотный тест — проверяет равномерность распределения единиц внутри блоков фиксированного размера.
- struct [RunsTest](#)
Тест на серии (runs test) — проверяет, соответствует ли количество серий (последовательностей одинаковых битов) случайному распределению.
- struct [LongestRunTest](#)
Тест на самую длинную серию — проверяет, не является ли максимальная длина серии в блоках слишком большой (что указывает на неслучайность).
- struct [CumulativeSumsTest](#)
Тест кумулятивных сумм — проверяет, насколько сильно отклоняется накопленная сумма (присваивая биту '1' +1, биту '0' -1) от нуля.

Functions

- double [erfc](#) (double x)
Дополнительная функция ошибок (erfc).

6.2.1 Function Documentation

6.2.1.1 erfc()

```
double nist::erfc (  
    double x)  [inline]
```

Дополнительная функция ошибок (erfc).

Parameters

x	аргумент.
---	-----------

Returns

1 - erf(x)

Глава 7

Class Documentation

7.1 nist::BlockFrequencyTest Struct Reference

Блочный частотный тест — проверяет равномерность распределения единиц внутри блоков фиксированного размера.

```
#include <nist_test.hpp>
```

Public Member Functions

- [BlockFrequencyTest](#) (const std::vector< bool > &bits, size_t blockSize)

Public Attributes

- double [p_value](#)

7.1.1 Detailed Description

Блочный частотный тест — проверяет равномерность распределения единиц внутри блоков фиксированного размера.

Последовательность разбивается на N непересекающихся блоков размера M . Для каждого блока вычисляется доля единиц pi_i . Статистика: $chi\text{-squared} = 4M \sum (pi_i - 0.5)^2$. Приблизённо распределена как $chi\text{-squared}$ с N степенями свободы. Для больших N используется нормальная аппроксимация.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 BlockFrequencyTest()

```
nist::BlockFrequencyTest::BlockFrequencyTest (
    const std::vector< bool > & bits,
    size_t blockSize) [inline]
```

Parameters

bits	входная последовательность.
blockSize	размер блока (в битах).

7.1.3 Member Data Documentation

7.1.3.1 p_value

```
double nist::BlockFrequencyTest::p_value
```

The documentation for this struct was generated from the following file:

- [nist_test.hpp](#)

7.2 nist::CumulativeSumsTest Struct Reference

Тест кумулятивных сумм — проверяет, насколько сильно отклоняется накопленная сумма (присваивая биту '1' +1, биту '0' -1) от нуля.

```
#include <nist_test.hpp>
```

Public Member Functions

- [CumulativeSumsTest](#) (const std::vector< bool > &bits)

Public Attributes

- double [p_value](#)

7.2.1 Detailed Description

Тест кумулятивных сумм — проверяет, насколько сильно отклоняется накопленная сумма (присваивая биту '1' +1, биту '0' -1) от нуля.

Вычисляются максимальное положительное и отрицательное отклонения $S_{\max}$ и $S_{\min}$. Нормированные значения: $z1 = S_{\max} / \text{sqrtn}$, $z2 = |S_{\min}| / \text{sqrtn}$. p-value = $P(z) = \sum_{k=-\infty}^{\infty} \{ \text{erfc}((2k+1)z/\text{sqrt2}) - \text{erfc}((2k-1)z/\text{sqrt2}) \}$. Окончательный p-value = $\min(P(z1), P(z2))$. Малое p-value означает, что накопленная сумма слишком часто (или слишком редко) пересекает нуль.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 CumulativeSumsTest()

```
nist::CumulativeSumsTest::CumulativeSumsTest (  
    const std::vector< bool > & bits) [inline]
```

7.2.3 Member Data Documentation

7.2.3.1 p_value

```
double nist::CumulativeSumsTest::p_value
```

The documentation for this struct was generated from the following file:

- [nist_test.hpp](#)

7.3 nist::FrequencyTest Struct Reference

Частотный (monobit) тест — проверяет, близка ли доля единиц во всей последовательности к 1/2.

```
#include <nist_test.hpp>
```

Public Member Functions

- [FrequencyTest](#) (const std::vector< bool > &bits)

Public Attributes

- double [p_value](#)

7.3.1 Detailed Description

Частотный (monobit) тест — проверяет, близка ли доля единиц во всей последовательности к 1/2.

Статистика: $S_n = (\text{количество единиц}) - (\text{количество нулей})$. Нормированная статистика: $S = |S_n| / \sqrt{n}$. p-value = $\text{erfc}(S/\sqrt{2})$. Если p-value $\geq \alpha$, последовательность случайна.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 FrequencyTest()

```
nist::FrequencyTest::FrequencyTest (
    const std::vector< bool > & bits) [inline]
```

Parameters

bits	входная битовая последовательность.
------	-------------------------------------

7.3.3 Member Data Documentation

7.3.3.1 p_value

```
double nist::FrequencyTest::p_value
```

The documentation for this struct was generated from the following file:

- [nist_test.hpp](#)

7.4 LCG_Standart Class Reference

Классический линейный конгруэнтный генератор (LCG) без модификаций.

```
#include <standart_generators.hpp>
```

Public Types

- using [result_type](#) = uint64_t

Public Member Functions

- [LCG_Standart](#) (uint64_t seed=123456789)
Конструктор.
- uint64_t [next](#) ()
Генерирует следующее 64-битное псевдослучайное число.

Static Public Member Functions

- static constexpr uint64_t [min](#) ()
- static constexpr uint64_t [max](#) ()

Private Attributes

- uint64_t [state](#)

Static Private Attributes

- static constexpr uint64_t [A](#) = 0x9e3779b97f4a7c15ULL
- static constexpr uint64_t [C](#) = 0xbf58476d1ce4e5b9ULL

7.4.1 Detailed Description

Классический линейный конгруэнтный генератор (LCG) без модификаций.

Формула: $state = A * state + C \bmod 2^{64}$. Параметры A и C выбраны так же, как в модифицированной версии, но без пост-перемешивания (XOR со сдвигами).

Note

Период: 2^{64} при правильных параметрах.

Warning

Младшие биты имеют короткий период, есть корреляции между соседними числами.

See also

[MLCG_XOR](#) – модифицированная версия с перемешиванием

7.4.2 Member Typedef Documentation

7.4.2.1 result_type

using [LCG_Standart::result_type](#) = uint64_t

7.4.3 Constructor & Destructor Documentation

7.4.3.1 LCG_Standart()

```
LCG_Standart::LCG_Standart (
    uint64_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор [LCG_Standart](#).

Parameters

seed	Начальное значение (по умолчанию 123456789)
seed	Начальное значение состояния.

Инициализирует внутреннее состояние заданным seed. Состояние будет обновляться по формуле LCG при каждом вызове [next\(\)](#).

7.4.4 Member Function Documentation

7.4.4.1 max()

`constexpr uint64_t LCG_Standart::max ()` [inline], [static], [constexpr]

7.4.4.2 min()

`constexpr uint64_t LCG_Standart::min ()` [inline], [static], [constexpr]

7.4.4.3 next()

`uint64_t LCG_Standart::next ()`

Генерирует следующее 64-битное псевдослучайное число.

Генерация следующего 64-битного числа.

Returns

Очередное значение LCG без пост-обработки.

Выполняется классический LCG шаг: $state = A * state + C$. Переполнение `uint64_t` автоматически даёт модуль 2^{64} .

Note

Отсутствие перемешивания приводит к корреляции младших битов.

7.4.5 Member Data Documentation

7.4.5.1 A

`uint64_t LCG_Standart::A = 0x9e3779b97f4a7c15ULL` [static], [constexpr], [private]

7.4.5.2 C

`uint64_t LCG_Standart::C = 0xbf58476d1ce4e5b9ULL` [static], [constexpr], [private]

7.4.5.3 state

`uint64_t LCG_Standart::state` [private]

The documentation for this class was generated from the following files:

- [standart_generators.hpp](#)
- [standart_generators.cpp](#)

7.5 LFG_Standart Class Reference

Классический генератор Фибоначчи с запаздыванием (LFG) без модификаций.

```
#include <standart_generators.hpp>
```

Public Types

- using [result_type](#) = uint32_t

Public Member Functions

- [LFG_Standart](#) (uint32_t seed=123456789)
Конструктор.
- uint32_t [next](#) ()
Генерирует следующее 32-битное псевдослучайное число.

Static Public Member Functions

- static constexpr uint32_t [min](#) ()
- static constexpr uint32_t [max](#) ()

Private Member Functions

- void [initBuffer](#) (uint32_t seed)
Инициализация буфера с заданным seed (простая версия без scramble).

Private Attributes

- uint32_t [buffer](#) [[LAG1](#)]
- int [index](#)

Static Private Attributes

- static constexpr int [LAG1](#) = 55
- static constexpr int [LAG2](#) = 24

7.5.1 Detailed Description

Классический генератор Фибоначчи с запаздыванием (LFG) без модификаций.

Формула: $buffer[i] = buffer[i] \wedge buffer[(i+24)\%55]$ (без scramble). Лаги: p=55, q=24.

Note

Период: $\sim 2^{79}$ для XOR-варианта.

Warning

Сильная зависимость от начального seed, возможны корреляции.

See also

[LFG_XOR](#) – модифицированная версия с scramble и улучшенной инициализацией

7.5.2 Member Typedef Documentation

7.5.2.1 result_type

using [LFG_Standart::result_type](#) = uint32_t

7.5.3 Constructor & Destructor Documentation

7.5.3.1 LFG_Standart()

```
LFG_Standart::LFG_Standart (
    uint32_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор [LFG_Standart](#).

Parameters

seed	Начальное значение (по умолчанию 123456789)
seed	Начальное значение для инициализации буфера.

Устанавливает index = LAG1 (55), чтобы при первом вызове [next\(\)](#) буфер был регенерирован. Вызывает `initBuffer(seed)` для заполнения начальных значений буфера.

7.5.4 Member Function Documentation

7.5.4.1 initBuffer()

```
void LFG_Standart::initBuffer (
    uint32_t seed) [private]
```

Инициализация буфера с заданным seed (простая версия без scramble).

Простая инициализация буфера без перемешивания.

Parameters

seed	Начальное значение.
------	---------------------

Заполняет буфер из 55 элементов с помощью LCG: $state = state * 1103515245 + 12345$.

Note

Нет функции `scramble`, нет дополнительных проходов перемешивания. Это делает генератор чувствительным к выбору seed.

7.5.4.2 max()

```
constexpr uint32_t LFG_Standart::max () [inline], [static], [constexpr]
```

7.5.4.3 min()

```
constexpr uint32_t LFG_Standart::min () [inline], [static], [constexpr]
```

7.5.4.4 next()

```
uint32_t LFG_Standart::next ()
```

Генерирует следующее 32-битное псевдослучайное число.

Генерация следующего 32-битного числа.

Returns

Очередное значение LFG без scramble.

Если индекс достиг конца буфера (55), весь буфер пересчитывается по формуле: $buffer[i] \wedge= buffer \leftarrow [(i+24)\%55]$. Затем возвращается текущий элемент буфера, и индекс увеличивается.

Note

Отсутствие scramble может приводить к корреляциям.

7.5.5 Member Data Documentation

7.5.5.1 buffer

```
uint32_t LFG_Standart::buffer[LAG1] [private]
```

7.5.5.2 index

```
int LFG_Standart::index [private]
```

7.5.5.3 LAG1

```
int LFG_Standart::LAG1 = 55 [static], [constexpr], [private]
```

7.5.5.4 LAG2

```
int LFG_Standart::LAG2 = 24 [static], [constexpr], [private]
```

The documentation for this class was generated from the following files:

- [standart_generators.hpp](#)
- [standart_generators.cpp](#)

7.6 LFG_XOR Class Reference

Генератор Фибоначчи с запаздыванием и XOR (LFG-XOR).

```
#include <modified_generators.hpp>
```

Public Types

- using [result_type](#) = uint32_t

Public Member Functions

- [LFG_XOR](#) (uint32_t seed=123456789)
Конструктор.
- uint32_t [next](#) ()
Генерирует 32-битное псевдослучайное число.

Static Public Member Functions

- static constexpr uint32_t [min](#) ()
Минимальное значение, возвращаемое [next\(\)](#).
- static constexpr uint32_t [max](#) ()
Максимальное значение, возвращаемое [next\(\)](#).

Private Member Functions

- uint32_t [scramble](#) (uint32_t x) const
Нелинейное перемешивание 32-битного слова (XOR + сдвиги).
- void [initBuffer](#) (uint32_t seed)
Инициализация буфера с заданным seed.

Private Attributes

- uint32_t [buffer](#) [[LAG1](#)]
Буфер состояний
- int [index](#)
Текущий индекс

Static Private Attributes

- static constexpr int [LAG1](#) = 55
Первый лаг
- static constexpr int [LAG2](#) = 24
Второй лаг

7.6.1 Detailed Description

Генератор Фибоначчи с запаздыванием и XOR (LFG-XOR).

Базовый алгоритм: LFG с лагами (55,24). Модификации:

- Вместо сложения используется XOR (быстрее, лучше перемешивание).
- Добавлена нелинейная функция scramble после обновлений.
- Улучшенная инициализация буфера через LCG и перемешивание.

7.6.2 Member Typedef Documentation

7.6.2.1 result_type

using [LFG_XOR::result_type](#) = uint32_t

7.6.3 Constructor & Destructor Documentation

7.6.3.1 LFG_XOR()

```
LFG_XOR::LFG_XOR (  
    uint32_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор [LFG_XOR](#).

Parameters

seed	Начальное значение (по умолчанию 123456789).
seed	начальное значение (по умолчанию 123456789).

7.6.4 Member Function Documentation

7.6.4.1 initBuffer()

```
void LFG_XOR::initBuffer (  
    uint32_t seed) [private]
```

Инициализация буфера с заданным seed.

Инициализация буфера генератора.

Parameters

seed	начальное значение для инициализации.
------	---------------------------------------

Алгоритм:

1. Заполняем буфер размера LAG1 (55) с помощью LCG ($state = state * 1103515245 + 12345$). Каждое значение сразу пропускается через `scramble` для начального перемешивания.
2. Затем 5 раз проходим по буферу: для каждого i берём элемент с отставанием $j = (i + LAG2) \% LAG1$, делаем XOR и снова `scramble`. Это гарантирует, что все начальные значения хорошо перемешаны и нет зависимости от `seed`.

7.6.4.2 `max()`

```
constexpr uint32_t LFG_XOR::max () [inline], [static], [constexpr]
```

Максимальное значение, возвращаемое `next()`.

7.6.4.3 `min()`

```
constexpr uint32_t LFG_XOR::min () [inline], [static], [constexpr]
```

Минимальное значение, возвращаемое `next()`.

7.6.4.4 `next()`

```
uint32_t LFG_XOR::next ()
```

Генерирует 32-битное псевдослучайное число.

Генерирует следующее 32-битное псевдослучайное число.

Returns

случайное число.

Если индекс достиг конца буфера ($index \geq LAG1$), то:

- проходим по всему буферу, для каждого i обновляем `buffer[i]` по правилу `buffer[i] ^= buffer[(i + LAG2) \% LAG1]`, затем применяем `scramble`.
- сбрасываем индекс на 0. Затем возвращаем `buffer[index++]` и увеличиваем индекс.

7.6.4.5 scramble()

```
uint32_t LFG_XOR::scramble (  
    uint32_t x) const    [private]
```

Нелинейное перемешивание 32-битного слова (XOR + сдвиги).

Нелинейное перемешивание 32-битного слова.

Parameters

x	входное 32-битное число.
---	--------------------------

Returns

перемешанное число.

Применяются три операции: сдвиг влево, сдвиг вправо и XOR.

7.6.5 Member Data Documentation

7.6.5.1 buffer

```
uint32_t LFG_XOR::buffer[LAG1]    [private]
```

Буфер состояний

7.6.5.2 index

```
int LFG_XOR::index    [private]
```

Текущий индекс

7.6.5.3 LAG1

```
int LFG_XOR::LAG1 = 55    [static], [constexpr], [private]
```

Первый лаг

7.6.5.4 LAG2

```
int LFG_XOR::LAG2 = 24    [static], [constexpr], [private]
```

Второй лаг

The documentation for this class was generated from the following files:

- [modified_generators.hpp](#)
- [modified_generators.cpp](#)

7.7 nist::LongestRunTest Struct Reference

Тест на самую длинную серию — проверяет, не является ли максимальная длина серии в блоках слишком большой (что указывает на неслучайность).

```
#include <nist_test.hpp>
```

Public Member Functions

- [LongestRunTest](#) (const std::vector< bool > &bits)

Public Attributes

- double [p_value](#)

7.7.1 Detailed Description

Тест на самую длинную серию — проверяет, не является ли максимальная длина серии в блоках слишком большой (что указывает на неслучайность).

Последовательность разбивается на блоки по 8 бит. Для каждого блока находится максимальная длина серии (подряд идущих одинаковых битов). Затем сравнивается наблюдаемое распределение максимальных длин с теоретическим. Статистика chi-squared рассчитывается по 7 категориям (длина le1, 2, 3, 4, 5, 6, ge7). p-value вычисляется приближённо как $\exp(-\text{chi-squared}/2)$.

7.7.2 Constructor & Destructor Documentation

7.7.2.1 LongestRunTest()

```
nist::LongestRunTest::LongestRunTest (  
    const std::vector< bool > & bits) [inline]
```

7.7.3 Member Data Documentation

7.7.3.1 p_value

```
double nist::LongestRunTest::p_value
```

The documentation for this struct was generated from the following file:

- [nist_test.hpp](#)

7.8 MLCG_XOR Class Reference

Модифицированный LCG с нелинейным перемешиванием (MLCG-XOR).

```
#include <modified_generators.hpp>
```

Public Types

- using `result_type` = `uint64_t`

Public Member Functions

- `MLCG_XOR` (`uint64_t` seed=123456789)
Конструктор.
- `uint64_t next` ()
Генерирует 64-битное псевдослучайное число.

Static Public Member Functions

- static constexpr `uint64_t min` ()
Минимальное значение, возвращаемое `next()`.
- static constexpr `uint64_t max` ()
Максимальное значение, возвращаемое `next()`.

Private Attributes

- `uint64_t state`
Текущее состояние

Static Private Attributes

- static constexpr `uint64_t A` = 0x9e3779b97f4a7c15ULL
Множитель (золотое сечение).
- static constexpr `uint64_t C` = 0xbf58476d1ce4e5b9ULL
Приращение (нечётное).

7.8.1 Detailed Description

Модифицированный LCG с нелинейным перемешиванием (MLCG-XOR).

Классический LCG: $X_{n+1} = (a * X_n + c) \bmod 2^{64}$. Модификации:

- Модуль 2^{64} за счёт естественного переполнения `uint64_t`.
- Параметры `a` и `c` подобраны для полного периода 2^{64} .
- После каждого шага применяется нелинейное перемешивание (XOR и сдвиги), устраняющее корреляцию младших битов.

7.8.2 Member Typedef Documentation

7.8.2.1 `result_type`

using `MLCG_XOR::result_type` = `uint64_t`

7.8.3 Constructor & Destructor Documentation

7.8.3.1 MLCG_XOR()

```
MLCG_XOR::MLCG_XOR (
    uint64_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор: сохраняет начальное состояние.

Parameters

seed	Начальное значение (по умолчанию 123456789)
seed	начальное значение (по умолчанию 123456789)

7.8.4 Member Function Documentation

7.8.4.1 max()

```
constexpr uint64_t MLCG_XOR::max () [inline], [static], [constexpr]
```

Максимальное значение, возвращаемое [next\(\)](#).

7.8.4.2 min()

```
constexpr uint64_t MLCG_XOR::min () [inline], [static], [constexpr]
```

Минимальное значение, возвращаемое [next\(\)](#).

7.8.4.3 next()

```
uint64_t MLCG_XOR::next ()
```

Генерирует 64-битное псевдослучайное число.

Генерирует следующее псевдослучайное 64-битное число.

Returns

64-битное число, равномерно распределённое в $[0, 2^{64}-1]$.

Шаги:

1. Выполняется LCG-шаг: $state = A * state + C$. Переполнение `uint64_t` автоматически берёт результат по модулю 2^{64} . Параметры A и C подобраны так, чтобы период был максимальным (2^{64}).
2. К полученному состоянию применяются три битовые операции:
 - $x \wedge = (x \gg 31)$ // сдвиг вправо и XOR
 - $x \wedge = (x \ll 25)$ // сдвиг влево и XOR
 - $x \wedge = (x \gg 28)$ // ещё один сдвиг вправо Эти операции нелинейны и хорошо перемешивают биты, разрушая корреляцию младших битов, свойственную LCG.

7.8.5 Member Data Documentation

7.8.5.1 A

uint64_t MLCG_XOR::A = 0x9e3779b97f4a7c15ULL [static], [constexpr], [private]

Множитель (золотое сечение).

7.8.5.2 C

uint64_t MLCG_XOR::C = 0xbf58476d1ce4e5b9ULL [static], [constexpr], [private]

Приращение (нечётное).

7.8.5.3 state

uint64_t MLCG_XOR::state [private]

Текущее состояние

The documentation for this class was generated from the following files:

- [modified_generators.hpp](#)
- [modified_generators.cpp](#)

7.9 nist::RunsTest Struct Reference

Тест на серии (runs test) — проверяет, соответствует ли количество серий (последовательностей одинаковых битов) случайному распределению.

```
#include <nist_test.hpp>
```

Public Member Functions

- [RunsTest](#) (const std::vector< bool > &bits)

Public Attributes

- double [p_value](#)

7.9.1 Detailed Description

Тест на серии (runs test) — проверяет, соответствует ли количество серий (последовательностей одинаковых битов) случайному распределению.

Пусть p_i = количество единиц / n . Если p_i не слишком отличается от 0.5, статистика: $Z = (R - 2np_i(1-p_i)) / (2\sqrt{2n}p_i(1-p_i))$, где R — общее число серий. $p\text{-value} = \text{erfc}(|Z|/\sqrt{2})$. Малое $p\text{-value}$ указывает на корреляцию соседних битов.

7.9.2 Constructor & Destructor Documentation

7.9.2.1 RunsTest()

```
nist::RunsTest::RunsTest (  
    const std::vector< bool > & bits) [inline]
```

7.9.3 Member Data Documentation

7.9.3.1 p_value

```
double nist::RunsTest::p_value
```

The documentation for this struct was generated from the following file:

- [nist_test.hpp](#)

7.10 Xorshift128_MLT Class Reference

Модифицированный Xorshift128 с нелинейным умножением (Xorshift128-MLT).

```
#include <modified_generators.hpp>
```

Public Types

- using [result_type](#) = uint64_t

Public Member Functions

- [Xorshift128_MLT](#) (uint64_t seed=123456789)
Конструктор.
- uint64_t [next](#) ()
Генерирует 64-битное псевдослучайное число.

Static Public Member Functions

- static constexpr uint64_t [min](#) ()
Минимальное значение, возвращаемое [next\(\)](#).
- static constexpr uint64_t [max](#) ()
Максимальное значение, возвращаемое [next\(\)](#).

Private Attributes

- uint64_t [s](#) [2]
Состояние (128 бит).

7.10.1 Detailed Description

Модифицированный Xorshift128 с нелинейным умножением (Xorshift128-MLT).

Базовый алгоритм: Xorshift128 (период $2^{128}-1$). Модификации:

- Добавлено нелинейное умножение на две константы и дополнительные XOR-сдвиги.
- Прогрев (10 итераций) для устранения корреляции начальных значений.

7.10.2 Member Typedef Documentation

7.10.2.1 result_type

using [Xorshift128_MLT::result_type](#) = uint64_t

7.10.3 Constructor & Destructor Documentation

7.10.3.1 Xorshift128_MLT()

```
Xorshift128_MLT::Xorshift128_MLT (
    uint64_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор [Xorshift128_MLT](#).

Parameters

seed	Начальное значение (по умолчанию 123456789).
seed	начальное значение (по умолчанию 123456789).

Инициализирует два 64-битных слова состояния: $s[0] = \text{seed}$, $s[1] = \text{seed} * \text{константа}$ (золотое сечение). Затем делается 10 итераций [next\(\)](#) для «прогрева» — чтобы начальные значения хорошо перемешались и убрать возможную корреляцию.

7.10.4 Member Function Documentation

7.10.4.1 max()

```
constexpr uint64_t Xorshift128_MLT::max () [inline], [static], [constexpr]
```

Максимальное значение, возвращаемое [next\(\)](#).

7.10.4.2 min()

```
constexpr uint64_t Xorshift128_MLT::min () [inline], [static], [constexpr]
```

Минимальное значение, возвращаемое [next\(\)](#).

7.10.4.3 next()

```
uint64_t Xorshift128_MLT::next ()
```

Генерирует 64-битное псевдослучайное число.

Генерирует следующее 64-битное псевдослучайное число.

Returns

случайное число.

Шаги:

1. Выполняется классический Xorshift128:

- берём $s1 = s[0]$, $s0 = s[1]$
- обновляем $s[0] = s0$
- выполняем серию XOR и сдвигов: $s1 \wedge= s1 \ll 23$; $s1 \wedge= s1 \gg 17$; $s1 \wedge= s0$; $s1 \wedge= s0 \gg 26$
- сохраняем $s[1] = s1$

2. Применяем нелинейное преобразование к s1:

- умножаем на константу 0xbf58476d1ce4e5b9ULL
- делаем XOR со сдвигом вправо на 31 бит
- умножаем на константу 0x94d049bb133111ebULL
- делаем XOR со сдвигом вправо на 27 бит Эти операции взяты из генератора SplitMix64 и обеспечивают высокое качество случайности, сохраняя период исходного Xorshift.

7.10.5 Member Data Documentation

7.10.5.1 s

```
uint64_t Xorshift128_MLT::s[2] [private]
```

Состояние (128 бит).

The documentation for this class was generated from the following files:

- [modified_generators.hpp](#)
- [modified_generators.cpp](#)

7.11 Xorshift128_Standart Class Reference

Классический Xorshift128 без нелинейного умножения.

```
#include <standart_generators.hpp>
```

Public Types

- using [result_type](#) = uint64_t

Public Member Functions

- [Xorshift128_Standart](#) (uint64_t seed=123456789)
Конструктор.
- uint64_t [next](#) ()
Генерирует следующее 64-битное псевдослучайное число.

Static Public Member Functions

- static constexpr uint64_t [min](#) ()
- static constexpr uint64_t [max](#) ()

Private Member Functions

- void [warmup](#) (int iterations=10)
Прогрев генератора (10 итераций для честного сравнения с модифицированной версией).

Private Attributes

- uint64_t [s](#) [2]

7.11.1 Detailed Description

Классический Xorshift128 без нелинейного умножения.

Формула: только XOR и сдвиги (23, 17, 26). Состояние: 128 бит (два 64-битных слова).

Note

Период: $2^{128} - 1$.

Warning

Линейность в GF(2) – по нескольким выходам можно восстановить состояние.

See also

[Xorshift128_MLT](#) – модифицированная версия с умножением (SplitMix64)

7.11.2 Member Typedef Documentation

7.11.2.1 result_type

using [Xorshift128_Standart::result_type](#) = uint64_t

7.11.3 Constructor & Destructor Documentation

7.11.3.1 Xorshift128_Standart()

```
Xorshift128_Standart::Xorshift128_Standart (
    uint64_t seed = 123456789) [explicit]
```

Конструктор.

Конструктор [Xorshift128_Standart](#).

Parameters

seed	Начальное значение (по умолчанию 123456789)
seed	Начальное значение.

Инициализирует состояние: $s[0] = \text{seed}$, $s[1] = \text{seed} * \text{золотое сечение}$. Затем выполняет прогрев (10 итераций) для честного сравнения с модифицированной версией [Xorshift128_MLT](#).

7.11.4 Member Function Documentation

7.11.4.1 max()

```
constexpr uint64_t Xorshift128_Standart::max () [inline], [static], [constexpr]
```

7.11.4.2 min()

```
constexpr uint64_t Xorshift128_Standart::min () [inline], [static], [constexpr]
```

7.11.4.3 next()

```
uint64_t Xorshift128_Standart::next ()
```

Генерирует следующее 64-битное псевдослучайное число.

Генерация следующего 64-битного числа.

Returns

Очередное значение Xorshift без нелинейного умножения.

Выполняется классический Xorshift128 с параметрами (23, 17, 26):

- $s1 \wedge = s1 \ll 23$
- $s1 \wedge = s1 \gg 17$
- $s1 \wedge = s0$
- $s1 \wedge = s0 \gg 26$

Note

Операции линейны в $\text{GF}(2)$, что может приводить к корреляциям. Модифицированная версия добавляет умножение для нелинейности.

7.11.4.4 warmup()

```
void Xorshift128_Standart::warmup (  
    int iterations = 10) [private]
```

Прогрев генератора (10 итераций для честного сравнения с модифицированной версией).

Прогрев генератора.

Parameters

iterations	Количество итераций прогрева.
------------	-------------------------------

Выполняет несколько итераций [next\(\)](#) без использования результатов. Это улучшает начальное состояние, особенно при плохих seed.

7.11.5 Member Data Documentation

7.11.5.1 s

```
uint64_t Xorshift128_Standart::s[2] [private]
```

The documentation for this class was generated from the following files:

- [standart_generators.hpp](#)
- [standart_generators.cpp](#)

Глава 8

File Documentation

8.1 chi_squar.hpp File Reference

Реализация критерия согласия хи-квадрат (Пирсона) для проверки равномерности распределения.

```
#include <vector>
#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <sstream>
```

Namespaces

- namespace [custom](#)

Enumerations

- enum class [custom::DistributionType](#) { [custom::UNIFORM_INT](#) , [custom::UNIFORM_REAL](#) }

Тип распределения: целочисленное равномерное или непрерывное равномерное.

Functions

- `template<typename Number>`
`std::pair< double, std::size_t > custom::chi\_squared (const std::vector< Number > &sample,`
`Number min, Number max, DistributionType type)`
Вычисляет статистику хи-квадрат и число степеней свободы для выборки.
- `template<typename Number>`
`bool custom::chi\_squared\_test (const std::vector< Number > &sample, Number min, Number`
`max, double significance, DistributionType type)`
Проверяет гипотезу о равномерном распределении выборки по критерию хи-квадрат.

8.1.1 Detailed Description

Реализация критерия согласия хи-квадрат (Пирсона) для проверки равномерности распределения.

Содержит функции `chi_squared` (вычисление статистики и степеней свободы) и `chi_squared_test` (проверка гипотезы на заданном уровне значимости).

8.2 chi_squar.hpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include <vector>
00011 #include <algorithm>
00012 #include <cmath>
00013 #include <stdexcept>
00014 #include <sstream>
00015
00016 namespace custom {
00017
00021
00022 enum class DistributionType {
00023     UNIFORM_INT,
00024     UNIFORM_REAL
00025 };
00026
00037
00038 template <typename Number>
00039 std::pair<double, std::size_t> chi_squared(const std::vector<Number>& sample,
00040     Number min, Number max,
00041     DistributionType type) {
00042     // Проверка корректности входных параметров
00043     if (min >= max) {
00044         std::ostringstream oss;
00045         oss << "Range minimum " << min << " is not less than range maximum " << max;
00046         throw std::invalid_argument(oss.str());
00047     }
00048     if (sample.size() < 2) {
00049         std::ostringstream oss;
00050         oss << "Sample size " << sample.size() << " is too small (need at least 2)";
00051         throw std::invalid_argument(oss.str());
00052     }
00053
00054     // Определяем длину интервала
00055     int add = (type == DistributionType::UNIFORM_INT) ? 1 : 0;
00056     double interval = static_cast<double>(max) - min + add;
00057
00058     // Количество интервалов (карманов) k = floor(log2(n)) + 1
00059     std::size_t num_buckets = static_cast<size_t>(std::log2(sample.size()) + 1);
00060     // Для целочисленных: если интервал меньше числа карманов, ограничиваем число возможных значений
00061     if (type == DistributionType::UNIFORM_INT && static_cast<double>(num_buckets) > interval)
00062         num_buckets = static_cast<size_t>(interval);
00063
00064     double bucket_size = interval / num_buckets; // ширина одного интервала
00065
00066     // Вычисляем границы интервалов
00067     std::vector<double> edges(num_buckets + 1);
00068     edges[0] = min;
00069     for (size_t i = 1; i <= num_buckets; ++i)
00070         edges[i] = min + bucket_size * i;
00071
00072     // Теоретические вероятности для каждого интервала (равномерное распределение)
00073     // = (ширина интервала)/interval
00074     std::vector<double> p_unif(num_buckets);
00075     for (size_t i = 0; i < num_buckets; ++i)
00076         p_unif[i] = (edges[i+1] - edges[i]) / interval;
00077
00078     // Наблюдаемые частоты: подсчёт количества элементов в каждом интервале
00079     // obs[i] — количество элементов выборки, попавших в i-й интервал
00080     std::vector<size_t> obs(num_buckets, 0);
00081     for (Number val : sample) {
00082         // Ищем позицию val в массиве границ (первый элемент, строго больший val)
00083         auto it = std::upper_bound(edges.begin(), edges.end(), val);
00084         size_t pos = std::distance(edges.begin(), it) - 1;
00085         // Защита от выхода за границы (если val точно равно правой границе последнего интервала)
00086         if (pos >= num_buckets) pos = num_buckets - 1;
00087         ++obs[pos];
00088     }
00089
00090     // Вычисление статистики хи-квадрат
00091     // chi-squared = sum (obs_i - exp_i)^2 / exp_i
00092     // Возвращаем chi-squared и степени свободы df = k - 1
00093     double chi = 0.0;
00094     for (size_t i = 0; i < num_buckets; ++i) {
00095         double expected = p_unif[i] * sample.size();
00096         double diff = obs[i] - expected;
00097         chi += diff * diff / expected;
00098     }
00099     return {chi, num_buckets - 1};
00100 }

```

```

00101
00112 template <typename Number>
00113 bool chi_squared_test(const std::vector<Number>& sample,
00114                      Number min, Number max, double significance,
00115                      DistributionType type) {
00116     // Уровень значимости должен быть строго между 0 и 1
00117     if (significance <= 0 || significance >= 1) {
00118         std::ostringstream oss;
00119         oss << "Significance level " << significance << " is not in (0; 1) interval";
00120         throw std::invalid_argument(oss.str());
00121     }
00122
00123     //Вычисляем chi-squared и df с помощью chi_squared
00124     auto [chi, df] = chi_squared(sample, min, max, type);
00125
00126     // Для больших df используем нормальную аппроксимацию (центральная предельная теорема)
00127     // chi-squared приблизительно нормально с параметрами df, 2df
00128     // Возвращаем true, если p-value > significance (или если chi-squared меньше критического значения)
00129     if (df > 30) {
00130         // Нормированное отклонение: (chi-squared - df) / sqrt(2df)
00131         double z = (chi - df) / std::sqrt(2 * df);
00132         // p-value = 1 - Phi(z) = 0.5 * erfc(z / sqrt(2))
00133         // Φ — функция распределения стандартного нормального закона
00134         double p = 0.5 * std::erfc(z / std::sqrt(2.0));
00135         return p > significance;
00136     } else {
00137         // Для малых df используем заранее вычисленные критические значения для уровня 0.05
00138         static const double critical[] = {3.84, 5.99, 7.81, 9.49, 11.07, 12.59, 14.07, 15.51, 16.92, 18.31};
00139         if (df <= 10)
00140             return chi <= critical[df - 1];
00141         else
00142             // Приближённая формула для df > 10 (до 30): chi-squared <= df + 2sqrt(2df)
00143             return chi <= df + 2 * std::sqrt(2 * df);
00144     }
00145 }
00146
00147 }

```

8.3 main.cpp File Reference

```

#include <iostream>
#include <iomanip>
#include <vector>
#include <numeric>
#include <cmath>
#include <algorithm>
#include <chrono>
#include <fstream>
#include <random>
#include "chi_squar.hpp"
#include "nist_test.hpp"
#include "modified_generators.hpp"
#include "standart_generators.hpp"

```

Functions

- template<typename T>
double **mean** (const std::vector< T > &sample)
Вычисляет выборочное среднее арифметическое.
- template<typename T>
double **stddev** (const std::vector< T > &sample)
Вычисляет выборочное стандартное отклонение.
- template<typename T>
double **variability** (const std::vector< T > &sample)
Вычисляет коэффициент вариации ($CV = stddev / mean$).

- `template<typename Generator>`
`std::vector< std::vector< typename Generator::result_type > > generateSamples (Generator &gen, size_t numSamples, size_t sampleSize)`
Генерирует заданное количество выборок указанного размера.
- `template<typename T>`
`void printStats (const std::string &name, const std::vector< std::vector< T > > &samples, T minVal, T maxVal, double significance)`
Выводит статистики и результат хи-квадрат для каждой выборки.
- `template<typename T>`
`std::vector< bool > toBits (const std::vector< T > &sample, int bitsPerNumber)`
Преобразует вектор целых чисел в битовую последовательность.
- `template<typename T>`
`void saveChiSquareSummary (const std::string &filename, const std::string &name, const std::vector< std::vector< T > > &samples, T minVal, T maxVal, double significance)`
Сохраняет сводку хи-квадрат тестов в CSV файл.
- `template<typename T>`
`void runNISTTests (const std::string &name, const std::vector< std::vector< T > > &samples, double significance)`
Запускает пять тестов NIST на объединённой битовой последовательности.
- `template<typename Generator>`
`std::vector< uint64_t > measureTimes (Generator &gen, const std::vector< size_t > &sizes)`
Замеряет время генерации заданного количества чисел.
- `void saveTimingsModified (const std::string &filename, const std::vector< size_t > &sizes, const std::vector< uint64_t > &t1, const std::vector< uint64_t > &t2, const std::vector< uint64_t > &t3, const std::vector< uint64_t > &t4)`
Сохраняет результаты замеров времени для модифицированных + mt19937_64.
- `void saveTimingsComparison (const std::string &filename, const std::vector< size_t > &sizes, const std::vector< uint64_t > &t1, const std::vector< uint64_t > &t2, const std::vector< uint64_t > &t3, const std::vector< uint64_t > &t4, const std::vector< uint64_t > &t5, const std::vector< uint64_t > &t6)`
Сохраняет результаты замеров времени для всех генераторов (модифицированные + стандартные).
- `template<typename Generator>`
`void saveCorrelationData (const std::string &filename, Generator &gen, size_t numPoints=1000)`
Сохраняет пары последовательных чисел для построения графика корреляций.
- `template<typename Generator>`
`void testSeedDependency (const std::vector< uint64_t > &seeds, const std::string &genName, size_t numSamples, size_t sampleSize, typename Generator::result_type minVal, typename Generator::result_type maxVal, double significance, const std::string &filename)`
Тестирует генератор с разными seed (включая плохие) и сохраняет результат хи-квадрат.
- `int main ()`

Variables

- `constexpr size_t NUM_SAMPLES = 20`
Количество выборок для статистического анализа
- `constexpr size_t SAMPLE_SIZE = 20000`
Размер каждой выборки (количество чисел).
- `constexpr double SIGNIFICANCE = 0.05`
Уровень значимости для статистических тестов ($\alpha = 0.05$).

8.3.1 Function Documentation

8.3.1.1 generateSamples()

```
template<typename Generator>
std::vector< std::vector< typename Generator::result_type > > generateSamples (
    Generator & gen,
    size_t numSamples,
    size_t sampleSize)
```

Генерирует заданное количество выборок указанного размера.

Template Parameters

Generator	Тип генератора (должен иметь метод next())
-----------	--

Parameters

gen	Генератор (передаётся по ссылке)
numSamples	Количество выборок
sampleSize	Размер каждой выборки

Returns

Вектор векторов сгенерированных чисел

8.3.1.2 main()

```
int main ()
```

8.3.1.3 mean()

```
template<typename T>
double mean (
    const std::vector< T > & sample)
```

Вычисляет выборочное среднее арифметическое.

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

sample	Вектор значений
--------	-----------------

Returns

Среднее арифметическое

8.3.1.4 measureTimes()

```
template<typename Generator>
std::vector< uint64_t > measureTimes (
    Generator & gen,
    const std::vector< size_t > & sizes)
```

Замеряет время генерации заданного количества чисел.

Template Parameters

Generator	Тип генератора
-----------	----------------

Parameters

gen	Генератор (передается по ссылке)
sizes	Вектор объемов выборок

Returns

Вектор времён в наносекундах

8.3.1.5 printStats()

```
template<typename T>
void printStats (
    const std::string & name,
    const std::vector< std::vector< T > > & samples,
    T minVal,
    T maxVal,
    double significance)
```

Выводит статистики и результат хи-квадрат для каждой выборки.

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

name	Имя генератора
samples	Вектор выборок
minVal	Минимальное значение генератора
maxVal	Максимальное значение генератора
significance	Уровень значимости

8.3.1.6 runNISTTests()

```
template<typename T>
void runNISTTests (
    const std::string & name,
    const std::vector< std::vector< T > > & samples,
    double significance)
```

Запускает пять тестов NIST на объединённой битовой последовательности.

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

name	Имя генератора
samples	Вектор выборок
significance	Уровень значимости

8.3.1.7 saveChiSquareSummary()

```
template<typename T>
void saveChiSquareSummary (
    const std::string & filename,
    const std::string & name,
    const std::vector< std::vector< T > > & samples,
    T minVal,
    T maxVal,
    double significance)
```

Сохраняет сводку хи-квадрат тестов в CSV файл.

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

filename	Имя файла
name	Имя генератора
samples	Вектор выборок
minVal	Минимальное значение генератора
maxVal	Максимальное значение генератора
significance	Уровень значимости

8.3.1.8 saveCorrelationData()

```
template<typename Generator>
void saveCorrelationData (
    const std::string & filename,
    Generator & gen,
    size_t numPoints = 1000)
```

Сохраняет пары последовательных чисел для построения графика корреляций.

Template Parameters

Generator	Тип генератора
-----------	----------------

Parameters

filename	Имя файла
gen	Генератор
numPoints	Количество точек

8.3.1.9 saveTimingsComparison()

```
void saveTimingsComparison (
    const std::string & filename,
    const std::vector< size_t > & sizes,
    const std::vector< uint64_t > & t1,
    const std::vector< uint64_t > & t2,
    const std::vector< uint64_t > & t3,
    const std::vector< uint64_t > & t4,
    const std::vector< uint64_t > & t5,
    const std::vector< uint64_t > & t6)
```

Сохраняет результаты замеров времени для всех генераторов (модифицированные + стандартные).

Parameters

filename	Имя файла
sizes	Объёмы выборок
t4	Векторы времён

8.3.1.10 saveTimingsModified()

```
void saveTimingsModified (
    const std::string & filename,
    const std::vector< size_t > & sizes,
    const std::vector< uint64_t > & t1,
    const std::vector< uint64_t > & t2,
```



```
const std::vector< uint64_t > & t3,
const std::vector< uint64_t > & t4)
```

Сохраняет результаты замеров времени для модифицированных + mt19937_64.

Parameters

filename	Имя файла
sizes	Объёмы выборок
t1	MLCG_XOR
t2	LFG_XOR
t3	Xorshift128_MLT
t4	mt19937_64

8.3.1.11 stddev()

```
template<typename T>
double stddev (
    const std::vector< T > & sample)
```

Вычисляет выборочное стандартное отклонение.

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

sample	Вектор значений
--------	-----------------

Returns

Стандартное отклонение

8.3.1.12 testSeedDependency()

```
template<typename Generator>
void testSeedDependency (
    const std::vector< uint64_t > & seeds,
    const std::string & genName,
    size_t numSamples,
    size_t sampleSize,
    typename Generator::result_type minVal,
    typename Generator::result_type maxVal,
    double significance,
    const std::string & filename)
```

Тестирует генератор с разными seed (включая плохие) и сохраняет результат хи-квадрат.

Template Parameters

Generator	Тип генератора
-----------	----------------

Parameters

seeds	Вектор seed-значений для тестирования
genName	Имя генератора (для вывода и имени файла)
numSamples	Количество выборок на каждый seed
sampleSize	Размер каждой выборки
minVal	Минимальное значение генератора
maxVal	Максимальное значение генератора
significance	Уровень значимости
filename	CSV-файл для сохранения результатов

8.3.1.13 toBits()

```
template<typename T>
std::vector< bool > toBits (
    const std::vector< T > & sample,
    int bitsPerNumber)
```

Преобразует вектор целых чисел в битовую последовательность.

Template Parameters

T	Целочисленный тип
---	-------------------

Parameters

sample	Вектор чисел
bitsPerNumber	Сколько младших битов использовать

Returns

Вектор битов (bool)

8.3.1.14 variability()

```
template<typename T>
double variability (
    const std::vector< T > & sample)
```

Вычисляет коэффициент вариации ($CV = \text{stddev} / \text{mean}$).

Template Parameters

T	Тип элементов выборки
---	-----------------------

Parameters

sample	Вектор значений
--------	-----------------

Returns

Коэффициент вариации

8.3.2 Variable Documentation

8.3.2.1 NUM_SAMPLES

```
size_t NUM_SAMPLES = 20 [constexpr]
```

Количество выборок для статистического анализа

8.3.2.2 SAMPLE_SIZE

```
size_t SAMPLE_SIZE = 20000 [constexpr]
```

Размер каждой выборки (количество чисел).

8.3.2.3 SIGNIFICANCE

```
double SIGNIFICANCE = 0.05 [constexpr]
```

Уровень значимости для статистических тестов ($\alpha = 0.05$).

8.4 modified_generators.cpp File Reference

Реализация трёх модифицированных генераторов псевдослучайных чисел.

```
#include "modified_generators.hpp"  
#include <cstdint>
```

8.4.1 Detailed Description

Реализация трёх модифицированных генераторов псевдослучайных чисел.

В этом файле реализованы три генератора:

- [MLCG_XOR](#): модифицированный LCG с нелинейным перемешиванием.
- [LFG_XOR](#): генератор Фибоначчи с запаздыванием и XOR.
- [Xorshift128_MLT](#): модифицированный Xorshift128 с нелинейным умножением.

8.5 modified_generators.hpp File Reference

Три модифицированных генератора псевдослучайных чисел.

```
#include <cstdint>
```

Classes

- class [MLCG_XOR](#)
Модифицированный LCG с нелинейным перемешиванием (MLCG-XOR).
- class [LFG_XOR](#)
Генератор Фибоначчи с запаздыванием и XOR (LFG-XOR).
- class [Xorshift128_MLT](#)
Модифицированный Xorshift128 с нелинейным умножением (Xorshift128-MLT).

8.5.1 Detailed Description

Три модифицированных генератора псевдослучайных чисел.

Содержит классы:

- [MLCG_XOR](#) (модифицированный LCG с нелинейным перемешиванием)
- [LFG_XOR](#) (генератор Фибоначчи с запаздыванием и XOR)
- [Xorshift128_MLT](#) (модифицированный Xorshift128 с умножением)

8.6 modified_generators.hpp

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011
00012 #include <stdint>
00013
00023 class MLCG_XOR {
00024 public:
00025     using result_type = uint64_t;
00026
00031     explicit MLCG_XOR(uint64_t seed = 123456789);
00032
00034     uint64_t next();
00035
00037     static constexpr uint64_t min() { return 0; }
00038
00040     static constexpr uint64_t max() { return ~0ULL; }
00041
00042 private:
00043     uint64_t state;
00044     static constexpr uint64_t A = 0x9e3779b97f4a7c15ULL;
00045     static constexpr uint64_t C = 0xbf58476d1ce4e5b9ULL;
00046 };
00047
00055 class LFG_XOR {
00056 public:
00057     using result_type = uint32_t;
00058
00063     explicit LFG_XOR(uint32_t seed = 123456789);
00064
00066     uint32_t next();
00067
00069     static constexpr uint32_t min() { return 0; }
00070
00072     static constexpr uint32_t max() { return ~0U; }
00073
00074 private:
00075     static constexpr int LAG1 = 55;
00076     static constexpr int LAG2 = 24;
00077     uint32_t buffer[LAG1];
00078     int index;
00079
00081     uint32_t scramble(uint32_t x) const;
00082
00084     void initBuffer(uint32_t seed);
00085 };
00086
00094 class Xorshift128_MLT {
00095 public:
00096     using result_type = uint64_t;
00097
00102     explicit Xorshift128_MLT(uint64_t seed = 123456789);
00103
00105     uint64_t next();
00106
00108     static constexpr uint64_t min() { return 0; }
00109
00111     static constexpr uint64_t max() { return ~0ULL; }
00112
00113 private:
00114     uint64_t s[2];
00115 };

```

8.7 nist_test.hpp File Reference

Реализация пяти статистических тестов из набора NIST SP 800-22.

```

#include <vector>
#include <cmath>
#include <algorithm>
#include <numeric>

```

Classes

- struct [nist::FrequencyTest](#)
Частотный (monobit) тест — проверяет, близка ли доля единиц во всей последовательности к $1/2$.
- struct [nist::BlockFrequencyTest](#)
Блочный частотный тест — проверяет равномерность распределения единиц внутри блоков фиксированного размера.
- struct [nist::RunsTest](#)
Тест на серии (runs test) — проверяет, соответствует ли количество серий (последовательностей одинаковых битов) случайному распределению.
- struct [nist::LongestRunTest](#)
Тест на самую длинную серию — проверяет, не является ли максимальная длина серии в блоках слишком большой (что указывает на неслучайность).
- struct [nist::CumulativeSumsTest](#)
Тест кумулятивных сумм — проверяет, насколько сильно отклоняется накопленная сумма (присваивая биту $1' + 1$, биту $0' - 1$) от нуля.

Namespaces

- namespace [nist](#)

Functions

- double [nist::erfc](#) (double x)
Дополнительная функция ошибок (erfc).

8.7.1 Detailed Description

Реализация пяти статистических тестов из набора NIST SP 800-22.

Набор тестов предназначен для проверки случайности битовых последовательностей. Реализованы следующие тесты:

- Frequency (Monobit) Test (проверка глобальной доли единиц)
- Block Frequency Test (проверка равномерности внутри блоков)
- Runs Test (проверка количества серий)
- Longest Run Test (проверка максимальной длины серии)
- Cumulative Sums Test (проверка случайности накопленной суммы)

Все тесты вычисляют p-value. Если p-value $\geq$ заданного уровня значимости, последовательность считается случайной.

8.8 nist_test.hpp

[Go to the documentation of this file.](#)

```

00001
00015
00016 # pragma once
00017
00018 #include <vector>
00019 #include <cmath>
00020 #include <algorithm>
00021 #include <numeric>
00022
00023 namespace nist {
00024
00030     inline double erfc(double x) {
00031         return 1 - std::erf(x);
00032     }
00033
00041     struct FrequencyTest {
00042         double p_value;
00043
00047         FrequencyTest(const std::vector<bool>& bits) {
00048             int64_t sum = 0;
00049             for (bool b : bits) sum += (b ? 1 : -1);
00050             double S = std::abs(sum) / std::sqrt(bits.size());
00051             p_value = erfc(S / std::sqrt(2.0));
00052         }
00053     };
00054
00063
00064     struct BlockFrequencyTest {
00065         double p_value;
00066
00071         BlockFrequencyTest(const std::vector<bool>& bits, size_t blockSize) {
00072             size_t M = blockSize;
00073             size_t N = bits.size() / M;
00074             if (N == 0) { p_value = 0; return; }
00075             double chi2 = 0.0;
00076             // Цикл проходит по каждому блоку по порядку
00077             for (size_t i = 0; i < N; ++i) {
00078                 size_t cnt = 0; // счётчик единиц в текущем блоке
00079                 // Внутренний цикл перебирает M битов текущего блока
00080                 for (size_t j = 0; j < M; ++j)
00081                     // Если бит равен 1 (истина), то cnt увеличивается
00082                     if (bits[i * M + j]) ++cnt;
00083                 double pi = static_cast<double>(cnt) / M; // выборочная доля единиц в текущем блоке
00084                 chi2 += (pi - 0.5) * (pi - 0.5);
00085             }
00086             chi2 *= 4.0 * M;
00087             // Для больших N используем нормальную аппроксимацию
00088             double mean = N;
00089             double var = 2 * N;
00090             double x = (chi2 - mean) / std::sqrt(var);
00091             p_value = erfc(std::abs(x) / std::sqrt(2.0));
00092         }
00093     };
00094
00102
00103     struct RunsTest {
00104         double p_value;
00105
00106         RunsTest(const std::vector<bool>& bits) {
00107             // Подсчитываем количество единиц во всей последовательности
00108             int64_t ones = std::count(bits.begin(), bits.end(), true);
00109             // Вычисляем выборочную долю единиц pi
00110             double pi = static_cast<double>(ones) / bits.size();
00111
00112             // Предварительная проверка: если pi сильно отклоняется от 0.5, тест не проводится
00113             // 2/sqrt(n) – это эмпирическое правило, используемое в NIST
00114             if (std::abs(pi - 0.5) > 2.0 / std::sqrt(bits.size())) {
00115                 p_value = 0.0;
00116                 return;
00117             }
00118
00119             // Начинаем подсчёт серий. Первая серия всегда существует, поэтому runs начинается с 1
00120             int64_t runs = 1;
00121             // Проходим по последовательности, начиная со второго бита
00122             for (size_t i = 1; i < bits.size(); ++i)
00123                 // Если текущий бит отличается от предыдущего, значит, началась новая серия – увеличиваем счётчик runs
00124                 if (bits[i] != bits[i-1]) ++runs;
00125             // Вычисляем числитель статистики
00126             double num = static_cast<double>(runs) - 2.0 * bits.size() * pi * (1.0 - pi);
00127             //Знаменатель
00128             double den = 2.0 * std::sqrt(2.0 * bits.size()) * pi * (1.0 - pi);
00129             p_value = erfc(std::abs(num) / den);

```

```

00130     }
00131 };
00132
00142 struct LongestRunTest {
00143     double p_value;
00144
00145     LongestRunTest(const std::vector<bool>& bits) {
00146         size_t blockSize = 8;
00147         size_t N = bits.size() / blockSize;
00148         std::vector<int> maxRuns(N, 0); // вектор для хранения максимальной длины серии единиц (run) в каждом
        блоке
00149
00150         // Для каждого блока i перебираются биты
00151         for (size_t i = 0; i < N; ++i) {
00152             int cur = 0, best = 0;
00153             for (size_t j = 0; j < blockSize; ++j) {
00154                 // Если бит равен 1, текущая длина cur увеличивается; если 0 – обнуляется
00155                 if (bits[i*blockSize + j]) cur++;
00156                 else cur = 0;
00157                 // В переменной best сохраняется максимальная длина серии единиц в этом блоке
00158                 if (cur > best) best = cur;
00159             }
00160             maxRuns[i] = best;
00161         }
00162         // Категории: 0–1, 2, 3, 4, 5, 6, ge7
00163         std::vector<int> obs(7, 0);
00164         for (int r : maxRuns) {
00165             if (r <= 1) obs[0]++;
00166             else if (r == 2) obs[1]++;
00167             else if (r == 3) obs[2]++;
00168             else if (r == 4) obs[3]++;
00169             else if (r == 5) obs[4]++;
00170             else if (r == 6) obs[5]++;
00171             else obs[6]++;
00172         }
00173         // Теоретические вероятности для блоков длиной 8 бит (взяты из документа NIST SP 800-22)
00174         std::vector<double> exp = {0.2148, 0.3672, 0.2305, 0.0977, 0.0312, 0.0084, 0.0502};
00175         double chi2 = 0.0;
00176         for (size_t i = 0; i < 7; ++i) {
00177             double e = exp[i] * N; // ожидаемое количество блоков в i-й категории
00178             // obs[i] – наблюдаемое
00179             chi2 += (obs[i] - e) * (obs[i] - e) / e;
00180         }
00181         // Приближённое p-value
00182         p_value = std::exp(-chi2 / 2);
00183     }
00184 };
00185
00195 struct CumulativeSumsTest {
00196     double p_value;
00197
00198     CumulativeSumsTest(const std::vector<bool>& bits) {
00199         int64_t S = 0; // текущая кумулятивная сумма
00200         int64_t sup = 0, inf = 0;
00201         // Цикл проходит по всем битам последовательности
00202         for (bool b : bits) {
00203             // если бит = 1, прибавляем +1; если 0, вычитаем 1
00204             S += (b ? 1 : -1);
00205             // Обновляем sup и inf, если текущее значение вышло за предыдущие границы
00206             if (S > sup) sup = S;
00207             if (S < inf) inf = S;
00208         }
00209         // Нормируем отклонения на квадратный корень из длины последовательности
00210         double z1 = std::abs(sup) / std::sqrt(bits.size()); // нормированный положительный максимум
00211         double z2 = std::abs(inf) / std::sqrt(bits.size()); // нормированный отрицательный минимум
00212
00213         // P(z) – это сумма по нечётным членам, дающая вероятность того, что максимум (по модулю) меньше z
00214         auto P = [](double z) {
00215             double sum = 0;
00216             // Суммирование по k от -5 до 5 даёт достаточную точность, так как члены с большими |k| пренебрежимо
        малы
00217             for (int k = -5; k <= 5; ++k) {
00218                 // разность дополнительных функций ошибок, которая эквивалентна разности нормальных CDF
00219                 double term = erfc((2*k+1)*z / std::sqrt(2.0)) - erfc((2*k-1)*z / std::sqrt(2.0));
00220                 sum += term;
00221             }
00222             return sum;
00223         };
00224         // итоговый p-value – это вероятность того, что при случайном блуждании и положительное, и
        отрицательное отклонения будут не больше наблюдаемых
00225         p_value = std::min(P(z1), P(z2));
00226     }
00227 };
00228
00229 }

```


8.9 standart_generators.cpp File Reference

```
#include "standart_generators.hpp"
```

8.10 standart_generators.hpp File Reference

Немодифицированные (классические) версии генераторов для сравнения.

```
#include <cstdint>
```

Classes

- class [LCG_Standart](#)
Классический линейный конгруэнтный генератор (LCG) без модификаций.
- class [LFG_Standart](#)
Классический генератор Фибоначчи с запаздыванием (LFG) без модификаций.
- class [Xorshift128_Standart](#)
Классический Xorshift128 без нелинейного умножения.

8.10.1 Detailed Description

Немодифицированные (классические) версии генераторов для сравнения.

Содержит классические реализации LCG, LFG и Xorshift128 без каких-либо нелинейных модификаций (перемешивание, умножение и т.д.).

Цель: сравнить статистические свойства и производительность модифицированных и немодифицированных версий.

8.11 standart_generators.hpp

[Go to the documentation of this file.](#)

```
00001 // standart_generators.hpp
00002 #ifndef STANDART_GENERATORS_HPP
00003 #define STANDART_GENERATORS_HPP
00004
00005 #include <cstdint>
00006
00017
00030 class LCG_Standart {
00031 public:
00032     using result_type = uint64_t;
00033
00038     explicit LCG_Standart(uint64_t seed = 123456789);
00039
00041     uint64_t next();
00042
00043     static constexpr uint64_t min() { return 0; }
00044     static constexpr uint64_t max() { return ~0ULL; }
00045
00046 private:
00047     uint64_t state;
00048     static constexpr uint64_t A = 0x9e3779b97f4a7c15ULL;
00049     static constexpr uint64_t C = 0xbf58476d1ce4e5b9ULL;
```

```
00050 };
00051
00063 class LFG_Standart {
00064 public:
00065     using result_type = uint32_t;
00066
00071     explicit LFG_Standart(uint32_t seed = 123456789);
00072
00074     uint32_t next();
00075
00076     static constexpr uint32_t min() { return 0; }
00077     static constexpr uint32_t max() { return ~0U; }
00078
00079 private:
00080     static constexpr int LAG1 = 55;
00081     static constexpr int LAG2 = 24;
00082     uint32_t buffer[LAG1];
00083     int index;
00084
00086     void initBuffer(uint32_t seed);
00087 };
00088
00100 class Xorshift128_Standart {
00101 public:
00102     using result_type = uint64_t;
00103
00108     explicit Xorshift128_Standart(uint64_t seed = 123456789);
00109
00111     uint64_t next();
00112
00113     static constexpr uint64_t min() { return 0; }
00114     static constexpr uint64_t max() { return ~0ULL; }
00115
00116 private:
00117     uint64_t s[2];
00118
00120     void warmup(int iterations = 10);
00121 };
00122
00123 #endif // STANDART_GENERATORS_HPP
```

Предметный указатель

A

- LCG_Standart, [22](#)
- MLCG_XOR, [33](#)

BlockFrequencyTest

- nist::BlockFrequencyTest, [18](#)

buffer

- LFG_Standart, [25](#)
- LFG_XOR, [29](#)

C

- LCG_Standart, [22](#)
- MLCG_XOR, [33](#)

chi_squar.hpp, [41](#)

chi_squared

- custom, [14](#)

chi_squared_test

- custom, [14](#)

CumulativeSumsTest

- nist::CumulativeSumsTest, [19](#)

custom, [13](#)

- chi_squared, [14](#)
- chi_squared_test, [14](#)
- DistributionType, [13](#)
- UNIFORM_INT, [13](#)
- UNIFORM_REAL, [13](#)

DistributionType

- custom, [13](#)

erfc

- nist, [16](#)

FrequencyTest

- nist::FrequencyTest, [20](#)

generateSamples

- main.cpp, [45](#)

index

- LFG_Standart, [25](#)
- LFG_XOR, [29](#)

initBuffer

- LFG_Standart, [24](#)
- LFG_XOR, [27](#)

LAG1

- LFG_Standart, [25](#)
- LFG_XOR, [29](#)

LAG2

- LFG_Standart, [25](#)

- LFG_XOR, [29](#)

LCG_Standart, [20](#)

- A, [22](#)

- C, [22](#)

- LCG_Standart, [21](#)

- max, [22](#)

- min, [22](#)

- next, [22](#)

- result_type, [21](#)

- state, [22](#)

LFG_Standart, [23](#)

- buffer, [25](#)

- index, [25](#)

- initBuffer, [24](#)

- LAG1, [25](#)

- LAG2, [25](#)

- LFG_Standart, [24](#)

- max, [24](#)

- min, [25](#)

- next, [25](#)

- result_type, [24](#)

LFG_XOR, [26](#)

- buffer, [29](#)

- index, [29](#)

- initBuffer, [27](#)

- LAG1, [29](#)

- LAG2, [29](#)

- LFG_XOR, [27](#)

- max, [28](#)

- min, [28](#)

- next, [28](#)

- result_type, [27](#)

- scramble, [28](#)

LongestRunTest

- nist::LongestRunTest, [30](#)

main

- main.cpp, [45](#)

main.cpp, [43](#)

- generateSamples, [45](#)

- main, [45](#)

- mean, [45](#)

- measureTimes, [45](#)

- NUM_SAMPLES, [51](#)

- printStats, [46](#)

- runNISTTests, [46](#)

- SAMPLE_SIZE, [51](#)

- saveChiSquareSummary, [47](#)

- saveCorrelationData, [47](#)

- saveTimingsComparison, [48](#)
- saveTimingsModified, [48](#)
- SIGNIFICANCE, [51](#)
- stddev, [49](#)
- testSeedDependency, [49](#)
- toBits, [50](#)
- variability, [50](#)
- max
 - LCG_Standart, [22](#)
 - LFG_Standart, [24](#)
 - LFG_XOR, [28](#)
 - MLCG_XOR, [32](#)
 - Xorshift128_MLT, [35](#)
 - Xorshift128_Standart, [38](#)
- mean
 - main.cpp, [45](#)
- measureTimes
 - main.cpp, [45](#)
- min
 - LCG_Standart, [22](#)
 - LFG_Standart, [25](#)
 - LFG_XOR, [28](#)
 - MLCG_XOR, [32](#)
 - Xorshift128_MLT, [35](#)
 - Xorshift128_Standart, [38](#)
- MLCG_XOR, [30](#)
 - A, [33](#)
 - C, [33](#)
 - max, [32](#)
 - min, [32](#)
 - MLCG_XOR, [32](#)
 - next, [32](#)
 - result_type, [31](#)
 - state, [33](#)
- modified_generators.cpp, [51](#)
- modified_generators.hpp, [52](#)
- next
 - LCG_Standart, [22](#)
 - LFG_Standart, [25](#)
 - LFG_XOR, [28](#)
 - MLCG_XOR, [32](#)
 - Xorshift128_MLT, [35](#)
 - Xorshift128_Standart, [38](#)
- nist, [15](#)
 - erfc, [16](#)
- nist::BlockFrequencyTest, [17](#)
 - BlockFrequencyTest, [18](#)
 - p_value, [18](#)
- nist::CumulativeSumsTest, [18](#)
 - CumulativeSumsTest, [19](#)
 - p_value, [19](#)
- nist::FrequencyTest, [19](#)
 - FrequencyTest, [20](#)
 - p_value, [20](#)
- nist::LongestRunTest, [30](#)
 - LongestRunTest, [30](#)
 - p_value, [30](#)
- nist::RunsTest, [33](#)
 - p_value, [34](#)
 - RunsTest, [34](#)
- nist_test.hpp, [53](#)
- NUM_SAMPLES
 - main.cpp, [51](#)
- p_value
 - nist::BlockFrequencyTest, [18](#)
 - nist::CumulativeSumsTest, [19](#)
 - nist::FrequencyTest, [20](#)
 - nist::LongestRunTest, [30](#)
 - nist::RunsTest, [34](#)
- printStats
 - main.cpp, [46](#)
- result_type
 - LCG_Standart, [21](#)
 - LFG_Standart, [24](#)
 - LFG_XOR, [27](#)
 - MLCG_XOR, [31](#)
 - Xorshift128_MLT, [35](#)
 - Xorshift128_Standart, [37](#)
- runNISTTests
 - main.cpp, [46](#)
- RunsTest
 - nist::RunsTest, [34](#)
- s
 - Xorshift128_MLT, [36](#)
 - Xorshift128_Standart, [39](#)
- SAMPLE_SIZE
 - main.cpp, [51](#)
- saveChiSquareSummary
 - main.cpp, [47](#)
- saveCorrelationData
 - main.cpp, [47](#)
- saveTimingsComparison
 - main.cpp, [48](#)
- saveTimingsModified
 - main.cpp, [48](#)
- scramble
 - LFG_XOR, [28](#)
- SIGNIFICANCE
 - main.cpp, [51](#)
- standart_generators.cpp, [57](#)
- standart_generators.hpp, [57](#)
- state
 - LCG_Standart, [22](#)
 - MLCG_XOR, [33](#)
- stddev
 - main.cpp, [49](#)
- testSeedDependency
 - main.cpp, [49](#)
- toBits
 - main.cpp, [50](#)
- UNIFORM_INT
 - custom, [13](#)

UNIFORM_REAL

custom, [13](#)

variability

main.cpp, [50](#)

warmup

Xorshift128_Standart, [38](#)

Xorshift128_MLT, [34](#)

max, [35](#)

min, [35](#)

next, [35](#)

result_type, [35](#)

s, [36](#)

Xorshift128_MLT, [35](#)

Xorshift128_Standart, [36](#)

max, [38](#)

min, [38](#)

next, [38](#)

result_type, [37](#)

s, [39](#)

warmup, [38](#)

Xorshift128_Standart, [38](#)